

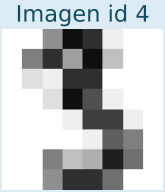
Aprendizaje Automático

Redes neuronales

20 de octubre de 2026

Eloy Alvarado Narváez

La pregunta que quedó abierta



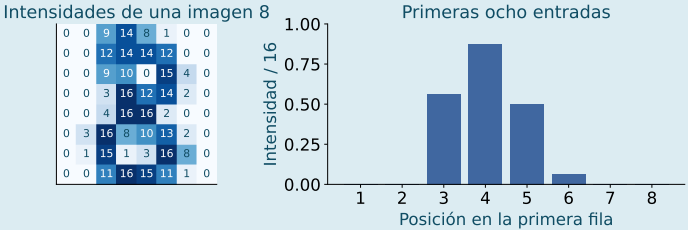




- $64 = 8 \times 8$ píxeles. 10 salidas: los dígitos de 0 a 9.
- Elegimos 32 unidades ocultas. Cada una calcula un valor: una activación.

El 32 es una decisión de arquitectura; lo compararemos con 16.

El caso de dígitos

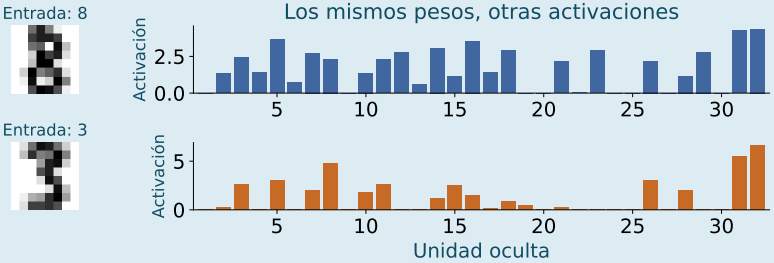


$$x_j = \frac{\text{intensidad}_j}{16}, \qquad \frac{14}{16} = 0.875, \qquad \frac{8}{16} = 0.5.$$

Leemos las filas en orden: $8 \times 8 = 64$ entradas. La etiqueta se guarda aparte.

Datos: UCI Optical Recognition of Handwritten Digits; copia load_digits.

Datos, parámetros y activaciones



- Pesos y sesgos: parámetros que ajustamos durante el entrenamiento.
- Activaciones: resultados que calculamos para cada imagen.

La misma red produce valores distintos al recibir imágenes distintas.

Una unidad de cálculo

$$a = w_1 x_1 + \cdots + w_p x_p + b, \qquad h = \phi(a).$$

- Una unidad o neurona recibe entradas x_j y tiene pesos w_j y un sesgo b.
- Primero calcula la suma a. Después aplica ϕ y entrega un número h.

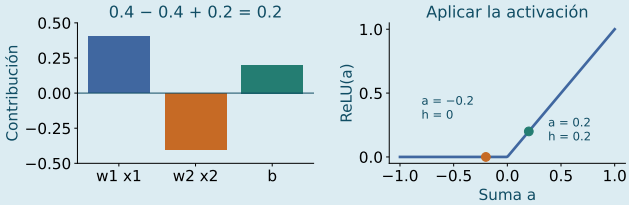
$$\mathrm{ReLU}(a) = \mathrm{máx}(0, a).$$

$$\mathrm{ReLU}(-2) = 0, \qquad \mathrm{ReLU}(3) = 3.$$

La activación es el resultado h de esa unidad para esa entrada.

Una unidad con números

$x = (0.5, 1), \quad w = (0.8, -0.4), \quad b = 0.2.$

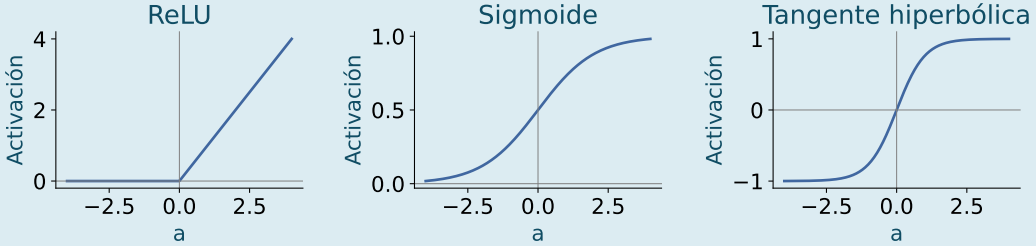


$$\text{Base: } a = 0.8(0.5) - 0.4(1) + 0.2 = 0.2, \quad h = 0.2.$$

$$x_2 = 2: \quad a = 0.4 - 0.8 + 0.2 = -0.2, \quad h = 0.$$

$$b = 0.5,\ x_2 = 1: \quad a = 0.4 - 0.4 + 0.5 = 0.5, \quad h = 0.5.$$

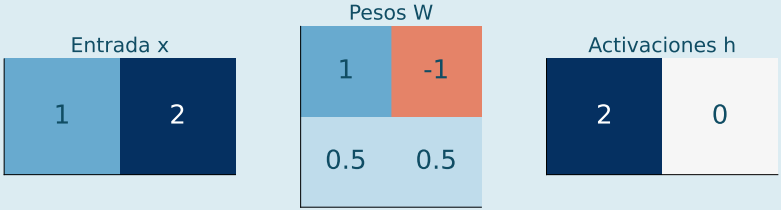
Funciones de activación



$$\mathrm{ReLU}(0) = 0, \qquad \sigma(0) = \frac{1}{1+e^{0}} = \frac{1}{2}, \qquad \tanh(0) = 0.$$

ReLU puede superar 1. La sigmoide queda entre 0 y 1; tanh, entre -1 y 1.

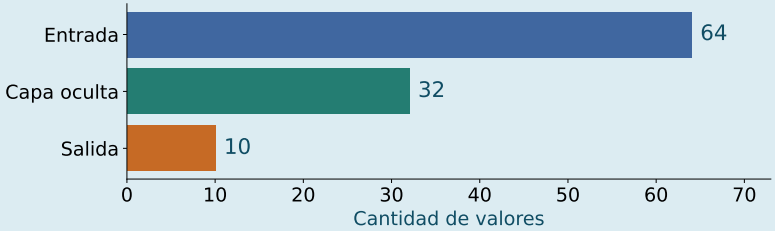
Qué es una capa



$$a_1 = 1(1) + 2(0.5) + 0 = 2, \qquad h_1 = \max(0, 2) = 2.$$
$$a_2 = 1(-1) + 2(0.5) + 0 = 0, \qquad h_2 = \max(0, 0) = 0.$$

Cada columna de pesos produce una activación. Aquí $h = (2, 0)$.

Leer una arquitectura: 64–32–10



- 64: intensidades de una imagen de 8×8 píxeles.
- 32: unidades ocultas elegidas antes del ajuste. Es un hiperparámetro.
- 10: clases posibles. Una probabilidad por dígito.

Compararemos anchos de 16 y 32 con validación.

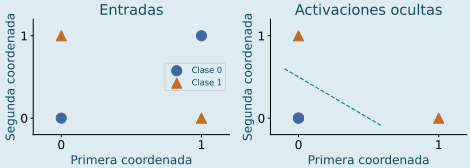
Por qué introducir no linealidad

$$h = xW_1 + b_1, \qquad s = hW_2 + b_2.$$

$$\begin{aligned} s &= (xW_1 + b_1)W_2 + b_2 \\ &= xW_1W_2 + b_1W_2 + b_2 \\ &= x\underbrace{(W_1W_2)}_{W_*} + \underbrace{(b_1W_2 + b_2)}_{b_*}. \end{aligned}$$

Sin una activación no lineal entre capas, obtenemos otra función afín.

Un ejemplo que necesita otra representación

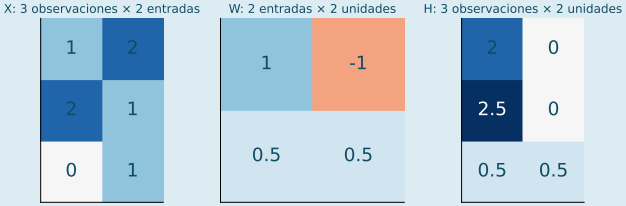


$h_1 = \text{máx}(0, x_1 - x_2)$, $h_2 = \text{máx}(0, x_2 - x_1)$.

(x_1, x_2)	$(x_1 - x_2, x_2 - x_1)$	(h_1, h_2)	$h_1 + h_2$	Clase
$(0, 0)$	$(0, 0)$	$(0, 0)$	0	0
$(1, 1)$	$(0, 0)$	$(0, 0)$	0	0
$(1, 0)$	$(1, -1)$	$(1, 0)$	1	1
$(0, 1)$	$(-1, 1)$	$(0, 1)$	1	1

Clase 1 si $h_1 + h_2 > 0.5$. Los pesos de este ejemplo se fijaron a mano.

Una capa procesa varias observaciones

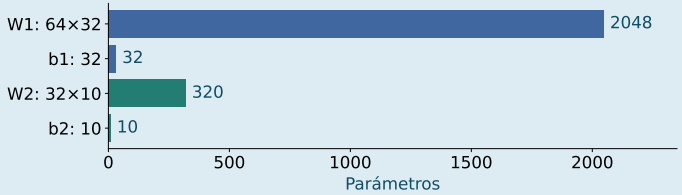


$$x = (2,1): \quad a = (2(1) + 1(0.5),\ 2(-1) + 1(0.5)) = (2.5, -1.5),$$
$$x = (0,1): \quad a = (0(1) + 1(0.5),\ 0(-1) + 1(0.5)) = (0.5, 0.5).$$

$$H = \mathrm{ReLU}(A) = \begin{pmatrix} 2 & 0 \\ 2.5 & 0 \\ 0.5 & 0.5 \end{pmatrix}.$$

$X_{3\times 2} W_{2\times 2} = A_{3\times 2}$. **Cada fila usa los mismos pesos y sesgos.**

Cuántos parámetros se ajustan



Ancho 32: $32(64+1)+10(32+1) = 2\,080 + 330 = 2\,410.$

Ancho 16: $16(64+1)+10(16+1) = 1\,040 + 170 = 1\,210.$

Contamos pesos y sesgos. Las activaciones y los píxeles no se suman al total.

Propagación: de la entrada a la capa oculta

$$x = (1, 2), \quad W_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 0.5 \end{pmatrix}, \quad b_1 = (0, 0).$$

$$a_{11} = 1(1) + 2(0.5) = 2, \qquad a_{12} = 1(-1) + 2(0.5) = 0.$$

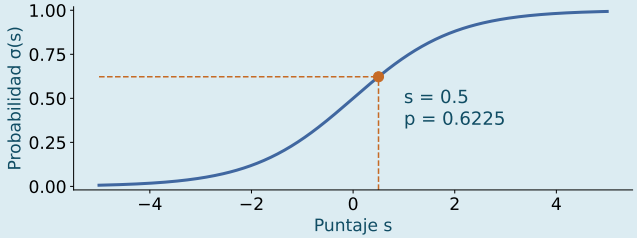
$$a_1 = (2, 0) \quad \Longrightarrow \quad h = \mathrm{ReLU}(a_1) = (2, 0).$$

Una columna de pesos produce una unidad. Aplicamos ReLU componente a componente.

Propagación: de la capa oculta a la probabilidad

$$h = (2, 0), \quad W_2 = (0.5, -1)^T, \quad b_2 = -0.5.$$

$$s = 2(0.5) + 0(-1) - 0.5 = 0.5.$$



$p = \sigma(0.5) = 1/(1 + e^{-0.5}) \approx 0.6225$. Ejemplo binario.

La salida depende de la tarea

Tarea	Salida habitual	Pérdida frecuente
Regresión	Un número real	Error cuadrático.
Clasificación binaria	Una sigmoide	Entropía cruzada binaria.
Clases excluyentes	Un softmax	Entropía cruzada multiclase.

La capa de salida y la pérdida deben corresponder a la respuesta.

Softmax convierte puntajes en probabilidades

$s = (0, \log 2, 0)$: exponenciar da $(1, 2, 1)$; la suma es 4.

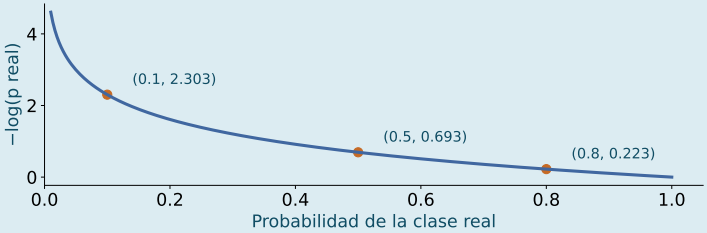


$$p = \left(\frac{1}{4}, \frac{2}{4}, \frac{1}{4}\right) = (0.25, 0.50, 0.25), \qquad \sum_k p_k = 1.$$

$$\frac{e^{s_k - c}}{\sum_j e^{s_j - c}} = \frac{e^{-c} e^{s_k}}{e^{-c} \sum_j e^{s_j}} = \frac{e^{s_k}}{\sum_j e^{s_j}}.$$

En el programa, $c = \max_j s_j$. Sumar 100 a todos equivale a $c = -100$.

La pérdida de una observación



$$\ell(y, p) = -\log p_y, \quad -\log(0.8) \approx 0.223, \quad -\log(0.5) \approx 0.693.$$
$$-\log(0.1) \approx 2.303.$$

Usamos la probabilidad de la clase real, aunque otra clase tenga mayor probabilidad.

El objetivo de entrenamiento

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, p_\theta(x_i)) + \frac{\lambda}{2}\sum_{l} \|W_l\|_F^2.$$

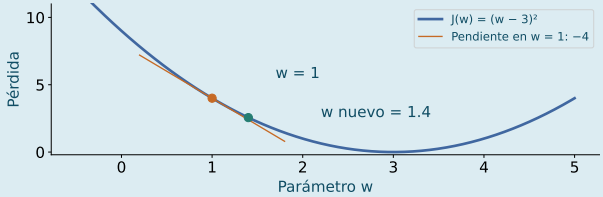
- θ reúne todos los pesos y sesgos.
- La primera parte mide ajuste a los datos.
- La penalización desalienta pesos grandes; aquí se escribe sin penalizar sesgos.

$$(-w)^2 = w^2, \qquad (2w)^2 = 4w^2.$$

Compararemos modelos con pérdida de validación sin penalización.

Qué indica un gradiente

$$J(w) = (w-3)^2, \quad J'(w) = 2(w-3), \quad \eta = 0.1.$$



$$J(1) = (1-3)^2 = 4, \qquad J'(1) = 2(1-3) = -4.$$

$$w^+ = 1 - 0.1(-4) = 1.4, \qquad J(1.4) = (1.4-3)^2 = 2.56.$$

Restar una derivada negativa aumenta el parámetro. Aquí disminuye la pérdida.

Un ejemplo completo para aprender

$x = 0.5, \quad y = 1, \quad (w_1, b_1, w_2, b_2) = (0.4, 0.1, 0.8, -0.2).$

$$
\begin{aligned}
a_1 &= w_1 x + b_1 = 0.4(0.5) + 0.1 = 0.3, \\
h &= \text{máx}(0, 0.3) = 0.3, \\
a_2 &= w_2 h + b_2 = 0.8(0.3) - 0.2 = 0.04, \\
p &= \frac{1}{1 + e^{-0.04}} \approx 0.509999, \\
\ell &= -\log(p) \approx 0.673347.
\end{aligned}
$$

Primero calculamos la predicción y la pérdida. Los cuatro parámetros siguen sin cambiar.

Derivar la pérdida en la salida

La clase de nuestro ejemplo es $y = 1$: $\ell = -\log p$.

$$\frac{d\ell}{dp} = -\frac{1}{p}, \qquad \frac{dp}{da_2} = p(1-p).$$

$$\delta_2 = \frac{d\ell}{da_2} = \left(-\frac{1}{p}\right) p(1-p) = p - 1 \approx -0.4900.$$

Si la clase fuera $y = 0$: $\ell = -\log(1-p)$.

$$\frac{d\ell}{da_2} = \frac{1}{1-p}\, p(1-p) = p.$$

Ambos casos: $\delta_2 = p - y$. Es la combinación de sigmoide y entropía cruzada binaria.

Retropropagación y regla de la cadena

$\delta_2 \approx -0.4900, \quad h = 0.3, \quad w_2 = 0.8, \quad x = 0.5.$

1. Parámetros de salida

$$\begin{aligned}\frac{\partial \ell}{\partial w_2} &= \delta_2 h = (-0.4900)(0.3) \\ &= -0.1470, \\ \frac{\partial \ell}{\partial b_2} &= \delta_2 = -0.4900.\end{aligned}$$

2. Parámetros de la capa oculta

$$\begin{aligned}\delta_1 &= \delta_2 w_2 \,\mathrm{ReLU}'(a_1) \\ &= (-0.4900)(0.8)(1) = -0.3920, \\ \frac{\partial \ell}{\partial w_1} &= \delta_1 x = (-0.3920)(0.5) = -0.1960, \\ \frac{\partial \ell}{\partial b_1} &= \delta_1 = -0.3920.\end{aligned}$$

$$\nabla \ell_{(w_1, b_1, w_2, b_2)} \approx (-0.1960, -0.3920, -0.1470, -0.4900).$$

Fin de la retropropagación: tenemos cuatro sensibilidades. El optimizador decidirá el cambio.

Un paso completo: resultado e interpretación

$\theta^+ = \theta - \eta \nabla \ell(\theta), \qquad \eta = 0.1.$

Actualizar los cuatro parámetros

$$w_1^+ \approx 0.4 - 0.1(-0.1960) = 0.4196,$$
$$b_1^+ \approx 0.1 - 0.1(-0.3920) = 0.1392,$$
$$w_2^+ \approx 0.8 - 0.1(-0.1470) = 0.8147,$$
$$b_2^+ \approx -0.2 - 0.1(-0.4900) = -0.1510.$$

Volver a calcular la predicción

$$a_1^+ = w_1^+(0.5) + b_1^+ \approx 0.3490,$$
$$h^+ = \max(0, a_1^+) \approx 0.3490,$$
$$a_2^+ = w_2^+ h^+ + b_2^+ \approx 0.1333,$$
$$p^+ = \sigma(a_2^+) \approx 0.533283.$$

$$\ell^+ = -\log(p^+) \approx 0.628702 < 0.673347.$$

Fin del paso: aumentó la probabilidad de la clase real y bajó su pérdida.

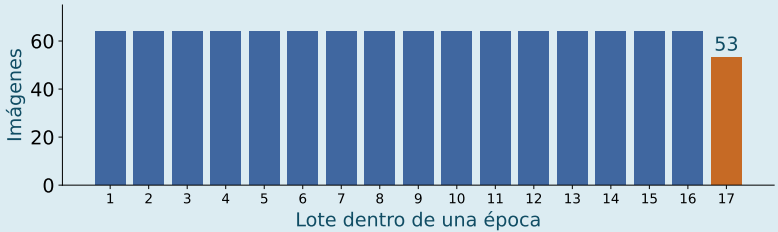
Con umbral 0.5, la clase sigue siendo 1. Este paso no demuestra generalización. Cifras redondeadas.

El ciclo de entrenamiento

- Calcular predicciones para un lote.
- Medir la pérdida y calcular gradientes.
- Actualizar parámetros con el optimizador.
- Repetir y observar el desempeño de validación.

Backpropagation calcula gradientes. El optimizador define la actualización.

Lote, actualización y época

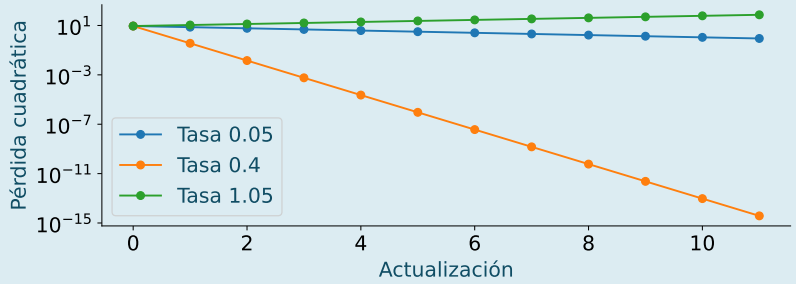


$$1\,077 = 16(64) + 53, \qquad 16 + 1 = 17 \text{ lotes por época.}$$

$$120 \text{ épocas} \times 17 \text{ actualizaciones} = 2\,040 \text{ actualizaciones.}$$

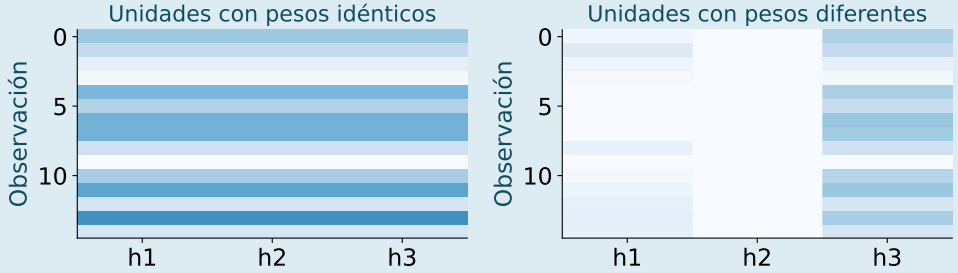
El último lote usa las 53 imágenes restantes de entrenamiento.

El tamaño del paso importa



Ejemplo didáctico: $L(w) = (w - 3)^2$, inicio $w = 0$. La escala vertical es logarítmica.

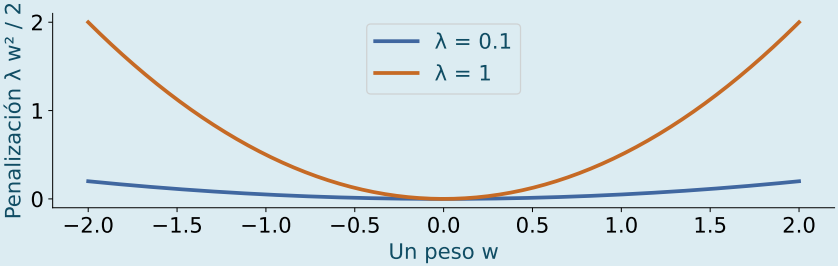
Inicialización y escala



Ejemplo didáctico, sin entrenamiento: unidades idénticas producen activaciones idénticas.

Inicialización, escala de entrada y semilla son decisiones diferentes.

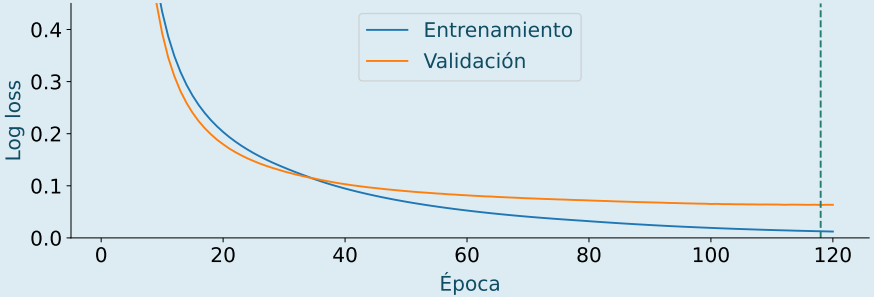
Capacidad y regularización



Penalización didáctica de un peso: $\lambda w^2/2$. No es una curva de validación.

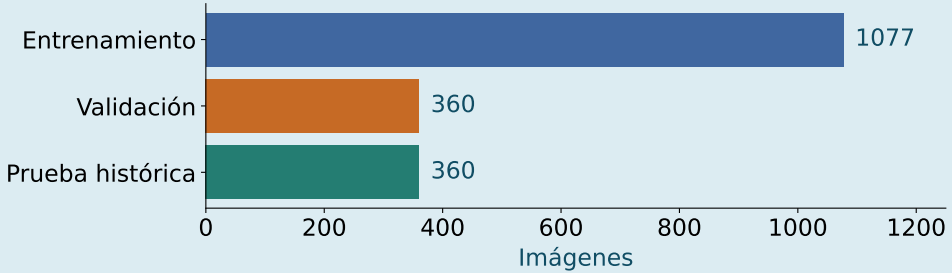
- Ancho y profundidad: qué funciones se pueden representar.
- Penalización y selección de época: cómo controlar el ajuste.

Cómo leer dos curvas



Pérdida predictiva sin penalización. La línea vertical marca la época elegida por validación.

Qué información puede usar cada etapa



Conservamos las imágenes y su partición. Los pesos elegidos por validación se mantienen sin reajustar.

La prueba histórica permite una comparación retrospectiva.

Entrenar y comparar en el computador

Secciones del programa	Operación y evidencia
00–02 · Comprender	Datos, propagación y una actualización manual.
03 · Configurar	Capas, activación, tasa, lote, penalización y épocas.
04 · Entrenar	Pesos nuevos; copia elegida por validación.
05–06 · Inspeccionar	Curvas y propagación con los pesos aprendidos.
07 · Evaluar	Prueba al fijar la receta; métricas y errores.

Cambiar una decisión a la vez y comparar con validación.

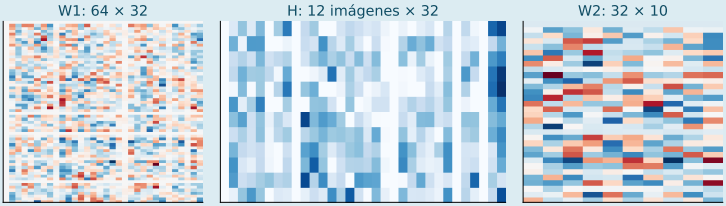
La receta base y sus parámetros

Elemento	Elección fijada
Candidatas	64–16–10 y 64–32–10.
Capas	ReLU oculta y softmax de salida.
Ajuste	Adam, tasa 0.002, lote 64, alpha 0.0001.
Presupuesto	120 épocas por candidata.
Selección	Menor log loss de validación, sin penalización.

Implementación: scikit-learn, modelos de redes neuronales supervisadas.

Cifras ilustradas: ejecución base de Python. R realiza un ajuste independiente.

Abramos la red que entrenamos

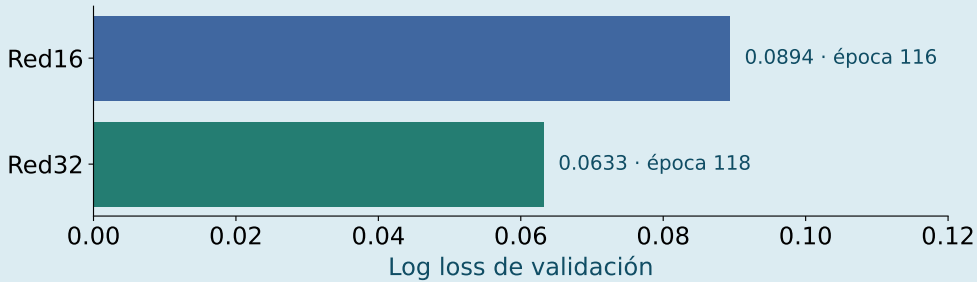


$$X_{360\times 64}\, W_{1,64\times 32} + b_{1,32} \xrightarrow{\text{ReLU}} H_{360\times 32}.$$

$$H_{360\times 32}\, W_{2,32\times 10} + b_{2,10} \xrightarrow{\text{softmax}} P_{360\times 10}.$$

Cada sesgo se suma a todas las filas. La figura muestra 12 imágenes; 06 procesa las 360 de validación.

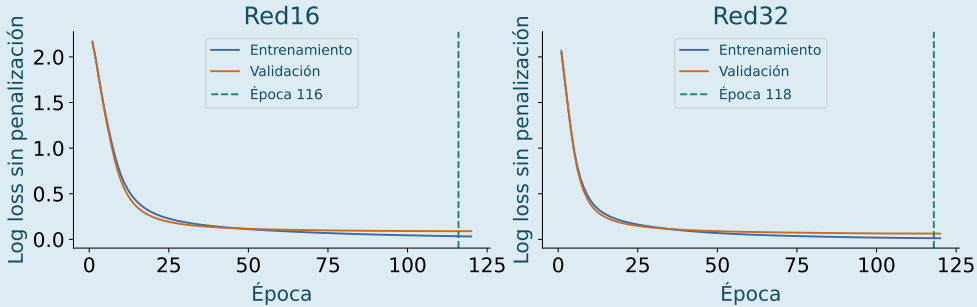
La elección por validación



Menor pérdida de validación observada en 120 épocas. Conservar Red32, época 118.

Mínimos cerca del final: no afirmamos convergencia.

El recorrido completo de las candidatas



Cada línea se calculó sobre el mismo conjunto a lo largo de las 120 épocas.

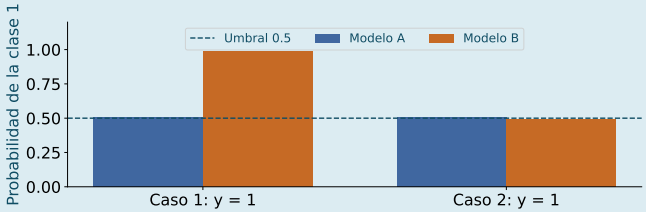
Comparación en la prueba histórica



Mismas 360 imágenes históricas. Exactitud: logística 97.78 %; red 97.22 %.

La red reduce log loss; la logística comete menos errores.

Por qué las métricas pueden discrepar

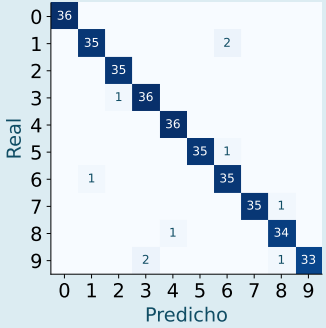


$$\bar{\ell}_A = \frac{-\log(0.51) - \log(0.51)}{2} \approx 0.6733 \qquad \text{(0 errores)}.$$

$$\bar{\ell}_B = \frac{-\log(0.99) - \log(0.49)}{2} \approx 0.3617 \qquad \text{(1 error)}.$$

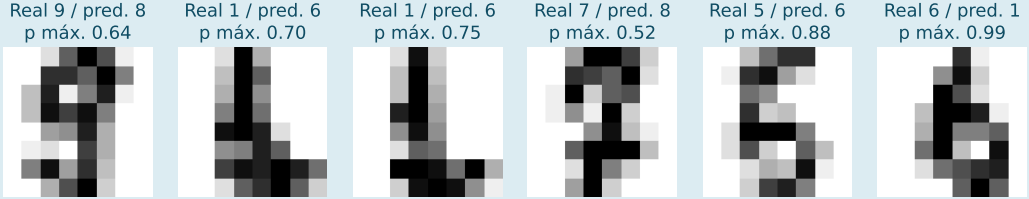
En ambos casos $y = 1$. El umbral 0.5 decide la clase; la pérdida evalúa las probabilidades.

Dónde se concentran los errores



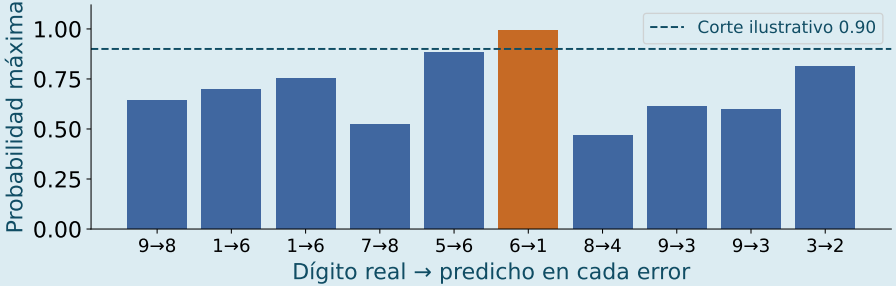
Filas: dígito real. Columnas: dígito predicho. Cada imagen aporta una unidad.

Inspección de imágenes mal clasificadas



Primeros seis errores según el orden de las filas de prueba; no representan todos los errores.

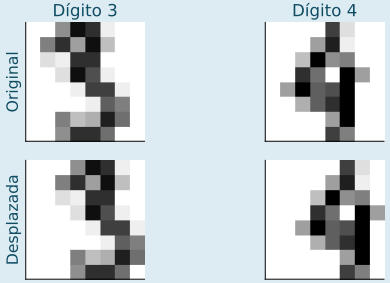
Una probabilidad alta también puede acompañar un error



Los 10 errores de la red en la prueba histórica. Etiquetas: dígito real $\rightarrow$ predicho.

Uno de los diez errores tiene probabilidad máxima de al menos 0.9.

La misma imagen en otra posición



Dos imágenes de prueba y un desplazamiento artificial de una columna hacia la derecha, rellenando con cero.

La clase se conserva en este ejercicio; las coordenadas que recibe la red cambian.

Alcance de esta comparación

- Una semilla y una cuadrícula pequeña no establecen estabilidad general.
- La prueba reutilizada permite una comparación retrospectiva.
- La separación por imagen no acredita transferencia a escritores nuevos.
- Una red densa no incorpora por sí sola invariancia a desplazamientos.

La evaluación debe corresponder a las condiciones reales de uso.

Diagnóstico antes de ampliar la red

Síntoma	Revisión razonable
Pérdida no finita	Datos, escala, tasa y estabilidad numérica.
No mejora entrenamiento	Gradientes, capacidad, tasa y objetivo.
Mejora solo entrenamiento	Regularización, época y datos.
Falla en otro contexto	Distribución, unidad de partición y evaluación.

Un procedimiento para discutir

- Se estandarizan todos los datos antes de separarlos.
- Se elige la época con mayor exactitud de prueba.
- Se comunica solo la pérdida de entrenamiento.
- Se concluye que la red generaliza a escritores nuevos.

Identifiquemos qué evidencia falta y qué decisión debemos corregir.

Una corrección del procedimiento

- Separar por la unidad relevante antes de estimar transformaciones.
- Ajustar escalado y parámetros con entrenamiento.
- Elegir receta y época mediante validación.
- Evaluar la receta fijada con datos apropiados para la afirmación.

La arquitectura y el protocolo de evaluación forman parte del mismo análisis.

Qué hace una red cuando aprende

$$x \xrightarrow{W_1,b_1} h \xrightarrow{W_2,b_2} p \xrightarrow{y} \ell.$$

$$\ell \xrightarrow{\text{regla de la cadena}} \nabla_\theta \ell \xrightarrow{\text{optimizador}} \theta^+.$$

- La propagación calcula una predicción.
- La retropropagación calcula sensibilidades.
- La actualización cambia los parámetros; validación orienta la selección.

Una pérdida menor en entrenamiento no equivale a una evaluación mejor.

El mismo mecanismo puede aprender otro objetivo

Clasificación: $x \longmapsto p(y \mid x)$.

Reconstrucción: $x \longmapsto h \longmapsto \widehat{x}$.

- Conservaremos capas, activaciones y gradientes.
- Cambiaremos la salida y la pérdida para reconstruir la entrada.
- Estudiaremos qué información conserva el código intermedio.

Próxima lectura: Goodfellow, Bengio y Courville, capítulo 14.